

Sistema de gestión del V Congreso Argentino de Agroecología

Trabajo Final Integrador — **Grupo 1**

De la maqueta al sistema desplegado: seis entregas, una aplicación en producción.

Integrantes: Benjamín Rodríguez Mantilla · Alcides Castro · Lucas Budnik

Jakarta EE 11

Jersey 3.1

Hibernate 7 · MySQL

Angular 19

Docker + GitLab CI

01

El problema

Qué congreso hay que gestionar, cómo nos acercamos y para quién construimos.

Un congreso no es un formulario: son cinco procesos en paralelo

2 min

El **V Congreso Argentino de Agroecología** (FCAyF–UNLP) reúne durante varios días a cientos de personas: docentes, productores, estudiantes y organizaciones. Hoy eso se coordina con planillas, mails y formularios sueltos.

Inscribirse y pagar

Categorías con aranceles distintos, transferencia o efectivo, comprobantes, recibos y facturas.

Enviar y evaluar trabajos

Resúmenes por eje temático, revisión por pares a ciegas, dictámenes, desempates y devoluciones.

Armar el programa

Mesas, pósters, talleres y conferencias en aulas con cupo y franjas horarias que no pueden solaparse.

Comunicar

Circulares, avisos por etapa y recordatorios: hoy, listas de correo copiadas a mano.

Certificar

Asistencia, presentación y evaluación. Al cierre, cada persona debe poder descargar lo suyo.

El costo de no tenerlo

Datos duplicados, plazos que se pasan, trabajos sin evaluador y certificados armados uno por uno.

Cómo nos acercamos al problema

- **Partimos del congreso real.** Del primer encuentro sobre el V Congreso Argentino de Agroecología 2027 y de lo documentado en ediciones anteriores (crecimiento de asistentes y trabajos, problemas con mails y planillas), para entender qué necesitaba realmente el sistema.
- **Primero un evento fijo; después plazos del comité.** Modelamos un congreso de tres días. Ahí vimos que casi todo lo que el sistema permite o bloquea depende del momento: inscripción, envío, evaluación, certificados. Pensando en otras ediciones, esos plazos los hicimos configurables: que dependan de la gestión del comité organizador, no de fechas fijas en el código.
- **De la organización real a roles, épicas e historias.** Vimos que el congreso necesitaba mejor organización porque hoy distintas personas hacen **tareas a mano** (inscribir y cobrar, enviar y evaluar trabajos, armar el programa, comunicar). Eso definió **roles concretos**, no un usuario genérico. Con esa base armamos la solución por **épicas** —es decir, grandes módulos del sistema: usuarios, inscripción, trabajos, cronograma, comunicación— y, dentro de cada una, **historias de usuario** (qué quiere hacer cada rol y para qué). Antes de programar las probamos en un prototipo navegable, solo de interfaz.

Usuarios y adoptantes de la solución

6 roles

Salieron de las tareas que hoy se hacen a mano: de ahí definimos roles y armamos la solución por épicas (módulos) e historias de usuario.

ROL	QUÉ HACE EN EL SISTEMA	POR QUÉ LE IMPORTA
Asistente	Se crea una cuenta e inicia sesión · se inscribe al congreso y completa sus datos · paga la inscripción (transferencia o efectivo) y sube el comprobante · consulta si su pago fue aprobado · arma su agenda personal del congreso sin chocar horarios · puede proponer un taller · descarga su certificado cuando el congreso termina.	Sabe en todo momento si su inscripción está bien, sin mandar mails para preguntar.
Autor	Además de lo del asistente: carga un trabajo (artículo o relato) con coautores y eje temático · sube el PDF · lo guarda como borrador o lo envía a evaluación · ve en qué estado está y lee la devolución · cuando lo aceptan, ve cuándo y dónde presenta.	Sigue su trabajo de punta a punta y recibe la devolución sin intermediarios.
Evaluador	Recibe trabajos de su eje temático · acepta o rechaza la asignación · lee el PDF · da su dictamen (aceptar, rechazar u observaciones) · puede adjuntar una corrección · no evalúa trabajos propios.	Solo recibe trabajos de su especialidad, con un cupo claro y sin conflicto de interés.
Comité académico (organizador científico)	Hace la prevalidación de los trabajos (revisión inicial antes de asignar evaluadores) · asigna evaluadores por eje temático respetando cupos · resuelve empates y reenvíos · aprueba solicitudes de nuevos evaluadores · configura ejes, modalidades y plazos de envío · consulta estadísticas del proceso académico.	Es el adoptante principal : hoy hace todo esto a mano.
Administración	Da de alta y gestiona usuarios · define aranceles (cuánto paga cada categoría de inscripción) · valida pagos, emite recibos/facturas y hace el arqueo de caja (control de lo cobrado en el día) · carga aulas, horarios y el programa del congreso · publica circulares y avisos · cierra el congreso y habilita certificados · ve reportes y pendientes de organización.	Centraliza la operación del evento y el control del dinero.

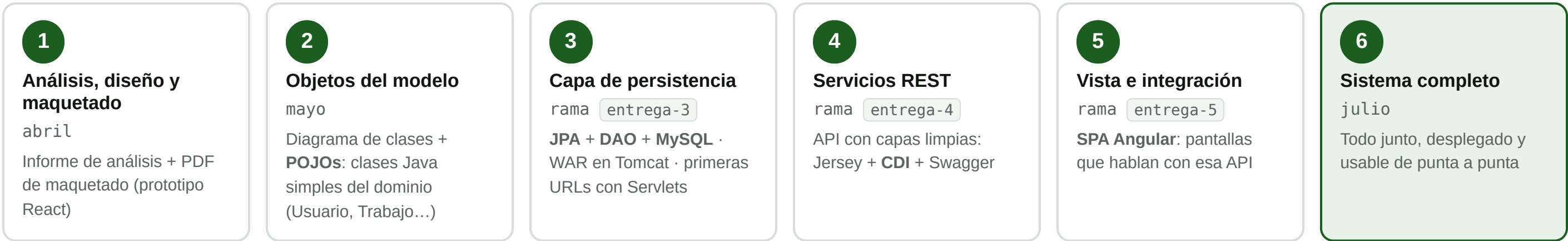
02

Las seis entregas

Qué objetivo tuvo cada etapa, qué conceptos de la cátedra aplicamos, con qué nos peleamos y qué quedó funcionando.

El camino: de la maqueta al sistema desplegado

abril → julio 2026



Una regla que nos salvó

Cada entrega vivió en su propia rama (entrega-3 , entrega-4 , entrega-5) y se integró a main sólo con el pipeline en verde. Nunca perdimos una entrega ya aprobada por trabajar sobre la siguiente.

Análisis, diseño y maquetado

Producto: informe + PDF de maquetado

Objetivo

Entender el dominio del V Congreso y acordar *qué* construir, con pantallas discutibles. Relevamiento → roles → **iniciativa** → **épicas** → **HU** → prototipo cliente–servidor (aún sin CDI).

① Análisis — HU más importantes para el congreso

- **HU-01 / HU-02** — Registrarse e iniciar sesión (entrada al sistema).
- **HU-06** — Solicitar inscripción y adjuntar comprobante de pago.
- **HU-07** — Admin valida pagos (aprobar/rechazar) y habilita al asistente.
- **HU-11** — Enviar un trabajo (PDF, eje, oral/póster) a evaluación.
- **HU-12** — Evaluador dictamina (aprobar/rechazar + devolución opcional).
- **HU-13 / HU-14** — Admin crea actividades y publica el programa.
- **HU-17** — Asistente arma su agenda personal sin solapar horarios.

Salen del informe *Análisis del Problema Sobre el V Congreso* (épicas 1–5).

Dificultad

Al maquetar aparecieron reglas que no estaban escritas: ¿qué pasa si dos evaluadores empatan?, ¿puede un evaluador revisar su propio trabajo?

Cómo la resolvimos: las anotamos en el análisis / HU y las llevamos al modelo de la Entrega 2.

② Maquetado: `archivo_maquetado.pdf` — capturas del prototipo **React + Vite** (`localStorage`, sin backend). Siguiente diapositiva: pantallas ligadas a esas HU.

Entrada al congreso: registro, inscripción y validación

archivo_maquetado.pdf

22/4/26, 13:13

Untitled

V Congreso Argentino de Agroecología

La Plata 2027

Iniciar sesión

Registrarse

o+

Registrarse

Crea tu cuenta para participar

Nombre

Apellido

Email

Contraseña

Confirmar contraseña

Registrarse

¿Ya tienes cuenta? Iniciar sesión

HU-01 Registro — crear cuenta para entrar al sistema

V Congreso Argentino de Agroecología

La Plata 2027

julieta

Inscripción al Congreso

Completá tus datos y adjuntá el comprobante de pago

Institución

Provincia

Selecciona

Categoría de Inscripción

Selecciona una categoría

Comprobante de Pago

Seleccionar archivo Sin archivos seleccionados

HU-06 Inscripción — datos + subir comprobante de pago

V Congreso Argentino de Agroecología

La Plata 2027

Admin

Panel de Administración

Gestión completa del congreso

Crear Mesa Temática

Crear Mesa Redonda

Crear Sesión de Pósters

Crear Taller

Crear Conferencia

Programa publicado

Controla si el cronograma es visible para el público en "Programa".

Publicado

Validación de Inscripciones

julieta lopez

Aprobar Rechazar

Ver comprobante

HU-07 Admin — ver comprobante, Aprobar / Rechazar

Trabajos, evaluación y programa

archivo_maquetado.pdf

V Congreso Argentino de Agroecología

1 julieta

La Plata 2027

Envío de Trabajos

Trabajos enviados: 1

Título

Eje temático

Oral

Poster

Adjunto (PDF obligatorio)

Seleccionar archivo Sin archivos seleccionados

El archivo se guarda localmente en este navegador para su revisión.

Enviar trabajo

HU-11 Envío — título, eje, oral/póster, PDF, Enviar

Evaluados

0

Aprobados

0

Trabajos Propuestos

Pendientes: 1

Red de Facilitadores en Agroecología

poster

Relato de experiencia

Pendiente

Ver / descargar PDF

Agregar comentario (opcional)

Aprobar

Rechazar

HU-12 Evaluador — PDF, comentario, Aprobar / Rechazar

PROGRAMA

Este es el calendario de actividades: mesas temáticas, mesas redondas, pósters y talleres.

Desde 10 de may de 2027 — hasta 12 de may de 2027

Ver programa completo

HU-13/14 Programa — calendario de actividades

Prototipo sin backend: validar el flujo de las HU antes de modelar en Java.

Entrega 1 · capturas 2/2

10 / 57

¿Cuál es nuestro diseño?

modelo de objetos del dominio

Diseñamos el congreso como **objetos Java del negocio** (POJOs): cada cosa importante es una clase con datos, estado y relaciones. Ese diseño **no guarda nada en MySQL todavía** — solo dice *qué* existe.

Ejemplo concreto

Trabajo

título, resumen, estado, PDF... métodos como `enviar()`.

AsignacionEvaluacion

¿este evaluador aceptó revisar este trabajo?

Evaluacion

el dictamen: aprobar / rechazar + comentario.

Lo mismo para `Usuario`, `Pago`, `InscripcionCongreso`, `Actividad` ... Más adelante mostramos ese diseño en el diagrama completo.

Qué problema deja planteado

Cuando el servidor se apaga, esos objetos en memoria **se pierden**.

Hace falta **persistencia**: copiar cada objeto a una fila de MySQL y volver a armarlo después.

Pregunta de diseño: ¿quién se ocupa de guardar y buscar? Si lo metemos en `Trabajo` o en el servlet, mezclamos el negocio con MySQL.

Por eso elegimos el patrón DAO: una responsabilidad aparte (acceso a datos de una entidad). Siguiendo: por qué ese camino.

Por qué fuimos por DAOs

separar “el negocio” de “la base”

El modelo de objetos ya estaba partido en entidades claras (`Usuario` , `Trabajo` ...). Eso nos empujó a un camino simple: **una entidad del diagrama → una tabla → un DAO**.

Camino que evitamos

La alternativa era escribir el código que habla con MySQL (`EntityManager` , consultas) directamente adentro de la clase `Usuario` o del servlet que atiende el pedido HTTP. Esto genera varios problemas:

- `Usuario` pasaría a depender de cómo está armada la base de datos: si mañana cambia algo en MySQL, hay que modificar la clase que define las reglas del usuario, aunque esas reglas no tengan nada que ver con la base.
- Si diez pantallas distintas necesitan guardar o buscar un usuario, la misma lógica de `persist` / `find` quedaría copiada diez veces, en vez de estar en un solo lugar.
- Se vuelve imposible probar que las reglas del negocio funcionan bien sin tener la base de datos levantada y conectada, porque quedan mezcladas en el mismo código.

En resumen, esta alternativa mezcla dos responsabilidades que deberían estar separadas: las reglas propias del congreso y la comunicación con MySQL.

Camino que elegimos: DAO

DAO = Data Access Object: es un **patrón de diseño** (no “magia de JPA”). En el código es una clase —o interfaz + implementación— cuya **única** responsabilidad es el acceso a datos de **una** entidad: guardar / buscar / actualizar / borrar.

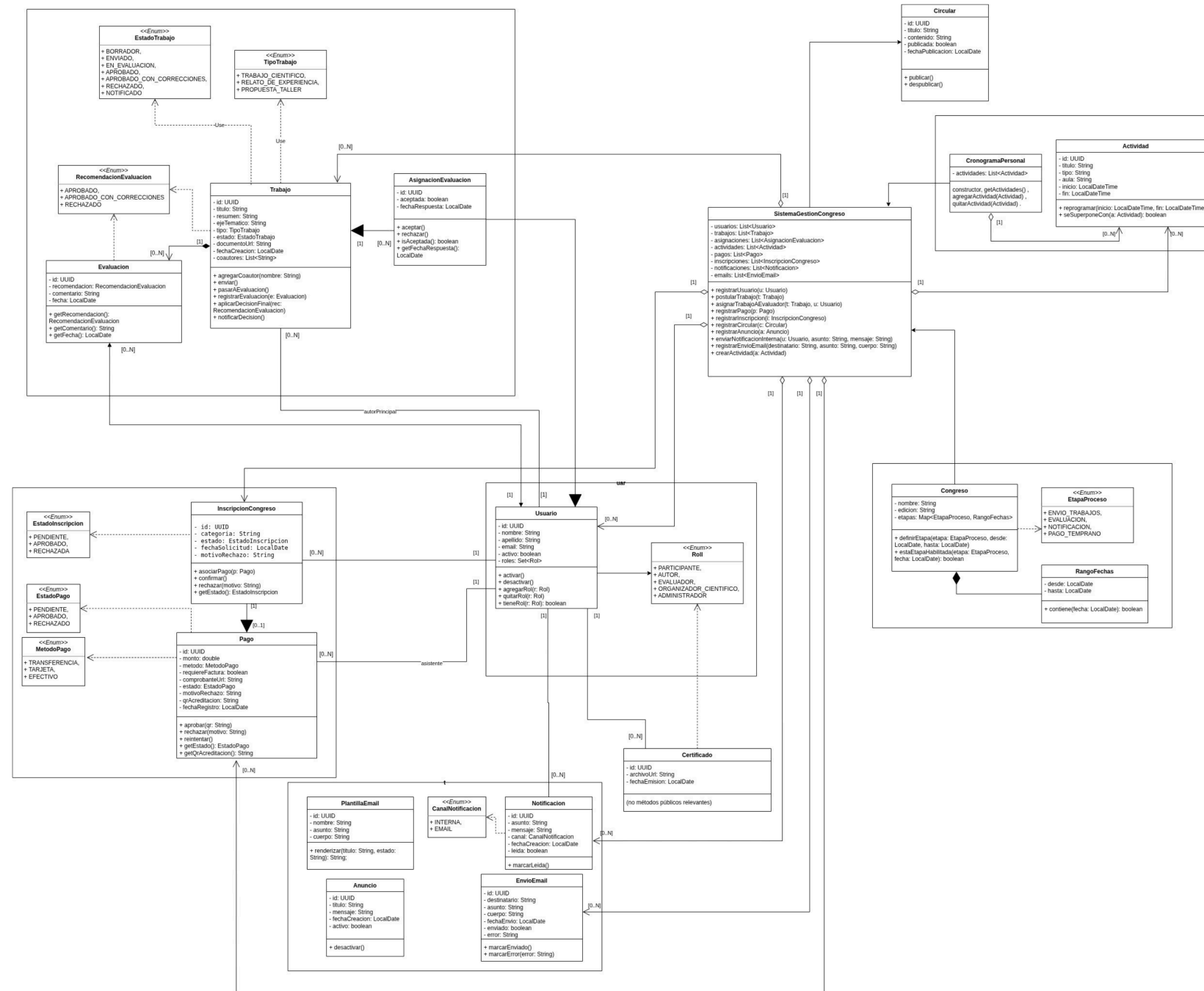
- `Usuario` = datos y reglas del usuario (modelo E2).
- `UsuarioDAO` + `UsuarioDAOImpl` = cómo entra y sale de MySQL (E3).
- **JPA** (`@Entity` , `EntityManager`) = la herramienta que usa el DAO para mapear objeto ↔ fila.

Por eso el modelo nos llevó a DAOs: ya teníamos objetos bien cortados; el DAO es el “portero” de cada uno hacia la base, sin ensuciar el diagrama de negocio.

Con el DAO ya separamos dominio e infraestructura. En la E3 el Servlet todavía hace de puerta HTTP y, a la vez, coordina el caso de uso. En la E4 terminamos ese corte: `Resource` → `Service` → `DAO` .

Camino del diseño

HU → clase del diagrama → POJO → entidad JPA → tabla MySQL → DAO (CRUD de esa entidad) → Factory (un solo lugar crea el DAO) → en E4, CDI inyecta esas dependencias.



Del diagrama a la persistencia (idea fija)

una caja → un DAO

Cuando hagan zoom en el diagrama, piensen esto: **cada clase con `id` es candidata a tabla y a DAO.**

EN EL MODELO (E2)	EN LA PERSISTENCIA (E3)	QUIÉN LO HACE
Usuario	tabla <code>usuario</code>	<code>UsuarioDAO</code>
Trabajo	tabla <code>trabajo</code>	<code>TrabajoDAO</code>
Pago	tabla <code>pago</code>	<code>PagoDAO</code>
AsignacionEvaluacion	su tabla	<code>AsignacionEvaluacionDAO</code>
Evaluacion	su tabla	<code>EvaluacionDAO</code>
CRUD repetido (save/find/delete)	código compartido	<code>GenericDAO</code> / <code>AbstractJpaDAO</code>
“¿quién me da el DAO?”	un solo lugar de creación	<code>DAOFactory</code> (Factory)

Nuestro diseño de objetos partió el dominio en entidades. Eso hizo natural el patrón **DAO**: cada entidad se persiste por su cuenta, sin meter MySQL dentro de las clases del negocio. **JPA** convierte el objeto en una fila de la tabla (y al revés). El **DAO** es quien pide ese guardar/buscar. El **Factory** es el único lugar que crea los DAOs: el resto del sistema no tiene que hacer `new UsuarioDAOImpl()` en cada pantalla.

Del modelo de objetos a datos que no se pierden

rama entrega-3

Venimos de la E2: ya teníamos el **diseño del dominio** (clases y relaciones). El objetivo de la E3 fue **persistirlo**: que un `Usuario` o un `Trabajo` viva en MySQL y sobreviva al reinicio del servidor.

Qué pedía la entrega

- Capa de persistencia con **JPA** sobre MySQL del curso.
- Patrón **DAO** con ABM (alta, baja, modificación, recuperación).
- App web Java desplegable (WAR / Tomcat) y verificable.

Hilo

Antes los datos vivían en el navegador (`localStorage`). Acá pasan al **servidor**. Todavía **no** es la API limpia con CDI (eso es E4). Primero había que demostrar que el modelo se guarda bien.

Para lograr esto necesitamos una **app web en Java**: un servidor que ejecute la aplicación, herramientas que conecten el modelo con la base, y un patrón que ordene el acceso a los datos. Empecemos por la base: **cómo se ejecuta la aplicación** (WAR + Tomcat).

Qué aprendimos a usar

qué hicimos · qué significa

En esta entrega no solo “guardamos datos”: aplicamos ideas centrales del desarrollo web en Java. Cada una responde a una pregunta distinta.

1 · WAR + Tomcat 10 (Jakarta)

La aplicación se **empaqueta en un solo archivo desplegable** (WAR). Un servidor de aplicaciones —Tomcat— es el encargado de ejecutarlo. Usamos la especificación moderna de Java para la web: **Jakarta EE** (Tomcat 10), que define cómo se escriben y despliegan este tipo de sistemas.

2 · Ciclo de vida (ServletContextListener)

Una app web no “está siempre prendida” de la misma forma: tiene un **ciclo de vida**. Al **arrancar**, se conecta a la base de datos y, si hace falta, carga datos iniciales para poder probar. Al **apagar**, se cierra todo de forma ordenada para no dejar conexiones ni recursos abiertos.

3 · Servlet

Cada funcionalidad importante tiene un **responsable** que atiende los pedidos que llegan por una dirección concreta (URL) y devuelve una respuesta según lo pedido: listar, crear, probar la persistencia, etc.

4 · Cliente–servidor + JSON

El **cliente** (navegador u otra herramienta) pide algo; el **servidor** responde. Esa respuesta puede ser **datos estructurados (JSON)**, pensados para que otro programa los use, o una **página** pensada para que la lea una persona (por ejemplo, el resultado de las pruebas de ABM).

5 · Persistencia orientada a objetos

En lugar de escribir a mano instrucciones de base de datos en cada pantalla, trabajamos con **objetos del dominio** (usuario, trabajo, pago...). Existe una **capa intermedia** que traduce esos objetos a datos guardados en tablas —y al revés al leerlos.

En una frase: empaquetamos y desplegamos la app → al arrancar queda lista la base → cada URL tiene quién la atiende → cliente y servidor se hablan → los objetos del modelo se persisten sin mezclar eso en cada pantalla.

Cómo las usamos, de punta a punta

no solo “cuál” · también “cómo”

No es una lista suelta: es la cadena que lleva lo que programamos hasta la app que corre en el servidor del curso.

Paso A · Armar el producto (Maven)

El código no se puede ejecutar tal cual en un servidor: hay que **empaquetarlo** en un solo archivo (WAR) con todo lo necesario. **Maven** arma ese paquete automáticamente: revisa qué librerías necesita la app (Hibernate, MySQL, Gson, Logback...), las descarga, compila y genera el WAR (`mvn package` → `jyaa2026-grupo1.war`).

Paso B · Hablar con la base (Hibernate + MySQL)

La app piensa en **objetos** (usuario, trabajo...); la base guarda **tablas**. **MySQL** es el motor que persiste esos datos en disco (`jyaa2026_bd1`). **Hibernate (JPA)** es el traductor automático entre ambos mundos: nosotros trabajamos con objetos; él genera el SQL. La configuración vive en `persistence.xml`.

Paso C · Responder al cliente (Gson + Logback)

El navegador no entiende objetos Java: hace falta un formato universal. **Gson** los traduce a **JSON** para que el cliente pueda usarlos. **Logback** es el “cuaderno de bitácora”: registra qué pasó (incluido el SQL) para detectar errores (`logback.xml`, consola de Tomcat).

Paso D · Llevarlo al servidor (Docker + GitLab CI)

En el servidor del curso no se instala cada proyecto a mano: se arma una **caja** (contenedor Docker) con Tomcat y el WAR listos para correr. **GitLab CI** automatiza ese proceso en un **pipeline**: cada vez que subimos código, se construye la imagen, se reemplaza el contenedor y **Traefik** publica la URL del grupo. Referencia: `Dockerfile` · dominio del curso.

Cómo el diagrama de la E2 se vuelve tablas

objeto Java ↔ fila MySQL

En la E2 el modelo vivía en memoria. En la E3 Hibernate se encarga de **guardarlo y recuperarlo** en MySQL, sin que escribamos el SQL a mano en cada pantalla.

¿Qué es una entidad?

En la E2 diseñamos clases Java que representaban cosas del congreso (`Trabajo`, `Usuario`), pero eran solo objetos en memoria. Una **entidad** es esa misma clase, marcada con `@Entity`, para decirle a Hibernate que debe guardarse en una tabla. El `@Id` identifica cada fila de forma única (como un DNI). Por ejemplo, `Trabajo` se convierte en la tabla de trabajos, y la relación con el autor (`@ManyToOne`) en una columna con el id de ese usuario (`autor_id`).

Relaciones = flechas del diagrama

El diagrama de la E2 mostraba relaciones (un trabajo tiene muchas evaluaciones, una inscripción un solo pago, etc.). En la base eso son **columnas que referencian otra fila**. Hibernate necesita el tipo de cada relación: `@OneToMany` / `@ManyToOne` (un trabajo, muchas evaluaciones; cada evaluación, un solo trabajo); `@OneToOne` (una inscripción, un único pago); `@ManyToMany` (actividad con varios trabajos y viceversa); `@ElementCollection` (listas simples: roles, coautores); `@Enumerated` (estados fijos como `EstadoTrabajo`).

`persistence.xml`

Es el archivo donde le decimos a Hibernate **cómo y dónde** conectarse: qué base, con qué usuario, y qué clases tratar como entidades. En Tomcat hay que **listar a mano** las ~15 clases: no las descubre solo.

EMF y EntityManager

El **EntityManagerFactory (EMF)** es la conexión general a la base: se crea **una sola vez** al arrancar (lo hace `JpaUtil`) y permanece abierta mientras corre el servidor.

El **EntityManager (EM)** ejecuta cada operación (guardar, buscar, actualizar, borrar): se abre para esa operación y se cierra apenas termina.

La **transacción**: si todo sale bien, `commit`; si falla a mitad, `rollback` (nada a medias).

Quién abre el EntityManager cada vez: el **DAO**. Quién lo crea: la **Factory**.

Quién guarda en MySQL (y por qué no el Servlet)

Si el Servlet hiciera SQL, mezclaríamos “atender pedidos HTTP” con “hablar con la base”. Por eso cada entidad tiene su DAO, y el Factory es el único lugar que crea esos DAOs.

DAO = Data Access Object (patrón)

El DAO es el “portero” de una sola entidad: el único responsable de guardar, buscar, actualizar o borrar sus datos (por ejemplo, todo lo de un trabajo pasa por `TrabajoDAO`). No es magia de JPA, sino una **decisión de diseño**: en código es una interfaz con su implementación (`TrabajoDAO` / `TrabajoDAOImpl`).

Cómo lo apilamos

Todas las entidades necesitan las mismas operaciones básicas (guardar, buscar, actualizar, borrar). En vez de repetir esa lógica, se define una sola vez: `GenericDAO<T>` declara el ABM común, y `AbstractJpaDAO` lo implementa (abre el EntityManager, arranca la transacción, ejecuta `persist` / `merge` / `remove` / `find`, y cierra). Cada entidad hereda eso y solo agrega consultas propias (por ejemplo, buscar por email).

Factory

Si cada Servlet o test creara el DAO con `new TrabajoDAOImpl()`, esa línea se repetiría en muchos archivos y un cambio de implementación obligaría a tocarlos todos.

`DAOFactory.getTrabajoDAO()` centraliza la creación: es un **patrón creacional**; todos piden el DAO ahí y no les importa cómo está armado por dentro.

Ejemplo completo: guardar un trabajo

1. Llega un pedido `POST /api/trabajos`: el Servlet lee el JSON del cliente.
2. Pide el DAO al Factory con `DAOFactory.getTrabajoDAO()` (y el de usuario si hace falta).
3. Arma el objeto `Trabajo` en memoria: todavía no está en la base.
4. Llama a `trabajoDAO.alta(t)`: ahí se abre la transacción, se guarda el objeto (`persist`) y se confirma el cambio (`commit`).
5. Hibernate genera el `INSERT` y MySQL asigna el `id`.
6. Gson arma la respuesta 201 con el trabajo ya persistido.

Prueba de la entrega: `/test-persistencia` ejecuta ~14 ABM (usuario, inscripción y pago, trabajo, asignación y evaluación...) y muestra OK o FALLO. Así respondemos la pregunta de la primera diapositiva: **quién guarda y busca** es el DAO, no el Servlet ni la clase del negocio. Con eso la E3 demuestra que el modelo ya vive en MySQL; en la siguiente vemos qué nos costó y qué quedó para la E4.

Qué nos costó y qué quedó para la E4

de la E3 a la E4

① “Unknown entity” en Tomcat

Hibernate se queja porque no reconoce una clase como entidad, aunque tenga `@Entity`. A diferencia de otros entornos, Tomcat no escanea el proyecto para descubrirlas solo. Lo resolvimos declarando cada clase a mano con una etiqueta `<class>` dentro de `persistence.xml`.

② Lazy loading al listar

Por defecto, algunas relaciones no se traen de la base hasta que se usan. Si se accede a esa colección **después** de cerrar la sesión, Hibernate falla (*could not initialize proxy — no Session*). `/test-persistencia` lo dejó en evidencia: ~13 ABM en OK y un FALLO en `Actividad.trabajos` (siguiente diapositiva). En el resto lo corregimos con `JOIN FETCH` en los DAOs; este caso pendiente muestra que la batería de pruebas sirve para detectar huecos reales.

③ Capas mezcladas (límite de la E3)

El Servlet pedía el DAO al Factory y manejaba el acceso a datos él mismo: mezclaba la puerta HTTP con el caso de uso y el acceso a MySQL. Alcanzó para demostrar el ABM, pero no es una API prolija. En la E4 separamos en tres capas — `Resource` (puerta HTTP), `Service` (aplicación / caso de uso) y `DAO` (infraestructura / MySQL) — dejando el dominio (`Usuario`, `Trabajo` ...) sin HTTP ni SQL. **CDI** (`@Inject`) las conecta y **Jersey** atiende HTTP.

Cierre: en la E3 el modelo **se persiste** y se despliega; la prueba ABM lo verifica en vivo. En la E4 ordenamos capas con **DIP**: la web ya no crea DAOs a mano — el contenedor los inyecta.

Pruebas ABM – DAO + JPA/Hibernate

HAY TESTS FALLIDOS

```
=== Tests ABM - Capa de persistencia (Grupo 1) ===
[Congreso (ABM + colección etapas)] OK
[Usuario (ABM + colección roles)] OK
[Pago (ABM)] OK
[InscripcionCongreso + Pago (ABM)] OK
[Trabajo (ABM + coautores)] OK
[Actividad (ABM + M2M trabajos)] FALLO – failed to lazily initialize a collection of role: Actividad.trabajos – could not initialize proxy - no Session
[Circular (ABM)] OK
[Notificacion (ABM)] OK
[EnvioEmail (ABM)] OK
[PlantillaEmail (ABM)] OK
[Anuncio (ABM)] OK
[Certificado (ABM)] OK
[CronogramaPersonal (ABM + M2M actividades)] OK
[AsignacionEvaluacion + Evaluacion (ABM)] OK → Datos DEMO guardados en BD (no se borran)

--- Resumen ---    OK: 13 | Fallos: 1
```

Misma salida que muestra el servlet en el navegador. El FALLO en `Actividad.trabajos` es el lazy loading del problema ② (sesión ya cerrada).

De la Práctica 6 al problema que resolvimos

rama entrega-4

Lo que ya sabíamos (Práctica 6) — el recorrido

1 · Del cliente a Jersey

El cliente manda un `POST` con JSON a `/api/usuarios`. Tomcat recibe el pedido y, como la URL empieza con `/api/*`, se lo pasa al `ServletContainer` de Jersey (registrado en `web.xml`). Jersey todavía no persiste nada: solo enruta según `@Path("/usuarios")`, `@POST`, `@Consumes` y `@Produces`.

2 · Jackson arma el objeto y se ejecuta `alta`

El método pide un parámetro `Usuario`, no un texto. Jersey busca un `MessageBodyReader`; encuentra el de **Jackson**, que parsea el JSON y construye el objeto. Recién ahí se ejecuta `UsuarioResource.alta(...)` con el usuario ya armado.

3 · El Resource le pasa el objeto al DAO

Desde `UsuarioResource` se recibe el objeto parseado y se lo entrega al DAO. El Resource no sabe SQL, no arma el JSON a mano y no abre la conexión a MySQL: solo atiende HTTP y delega la persistencia.

4 · Hibernate traduce el objeto a un `INSERT`

Al arrancar, `JpaBootstrapListener` crea una sola vez la `EntityManagerFactory`. En cada operación el DAO obtiene un `EntityManager` y llama a `em.persist(usuario)`. Hibernate lee `@Entity` y `@Table` de la clase y arma el `INSERT` que ejecuta MySQL.

5 · La respuesta vuelve en JSON

El método hace `return` del resultado. Jersey lee `@Produces`, busca un `MessageBodyWriter` y Jackson convierte el objeto otra vez a JSON. El cliente recibe HTTP 200 (o el código que corresponda) con ese body. La Entrega 4 no tira abajo este recorrido: lo extiende.

El problema

En la Práctica 6, `UsuarioResource` dependía de la clase concreta `UsuarioDAOImpl`. Si cambiábamos cómo se persiste, había que tocar la capa HTTP. Esa capa no debería saber quién fabrica el DAO ni cómo habla con MySQL.

Qué pedía la E4: separar en capas, que el Resource deje de conocer `UsuarioDAOImpl`, y que **CDI / Weld** conecte las piezas con `@Inject`.

Cuatro capas, una responsabilidad cada una

presentación · servicio · dominio · infraestructura

Presentación

Recibe el pedido HTTP y arma la respuesta (201, 400, JSON). Ejemplo: `UsuarioResource`. No sabe SQL ni cómo se abre la conexión a la base.

Dominio

Qué es cada cosa del negocio: `Usuario`, `Trabajo`, estados y roles. Sin HTTP y sin detalles de MySQL.

Aplicación / Servicio

Coordina el caso de uso: valida reglas y recién ahí pide guardar o buscar. Ejemplo: `UsuarioService` (email único, rol por defecto).

Infraestructura

Habla con el mundo técnico: la implementación del DAO, Hibernate, MySQL, y el almacenamiento de archivos PDF en disco.

Qué sí y qué no puede saber `UsuarioResource`

La razón de fondo: cada capa solo conoce a la de al lado, nunca a las de más abajo. El Resource habla con el Service; el Service habla con la interfaz del DAO; la implementación del DAO habla con MySQL. Si el Resource conociera `UsuarioDAOImpl`, un cambio en cómo se guarda obligaría a tocar la puerta HTTP. Por eso el Resource no “salta” capas.

Sí: que existe `UsuarioService`, que puede pedirle el alta de un usuario, códigos HTTP, y que el cuerpo del pedido es un usuario o un DTO.

No: que la implementación se llama `UsuarioDAOImpl`, nombres de tablas, ni cómo se abre o cierra la sesión con la base.

DTOs: entrada y salida

Un **DTO** (Data Transfer Object) es un objeto pensado solo para viajar por la red: decide exactamente qué datos entran o salen en el mensaje. Si devolvemos la entidad completa del usuario, mandamos de más: la contraseña, y relaciones entre objetos que pueden hacer que la respuesta se enrede en un ciclo infinito. El DTO corta ese problema: controlamos qué viaja en el mensaje.

En el proyecto hay dos casos concretos de esa idea. **Entrada:** en el login solo hacen falta el email y la contraseña; no hace falta mandar todo el usuario. **Salida:** cuando respondemos con los datos de un usuario, el mensaje no incluye la contraseña.

A futuro, separar el DTO de la entidad protege el contrato con quien consume la API (Swagger, el frontend, otro cliente). Si el modelo de usuario cambia o crece en próximas entregas, ese cambio queda contenido en el dominio: no se filtra solo al JSON que ya reciben los clientes. Quien decide qué se muestra es el DTO, no la entidad.

Si el Resource no conoce el DAO concreto, ¿a qué le habla?

la pregunta que quedó abierta

La pregunta que quedó de la diapositiva anterior

Dijimos que `UsuarioResource` no puede conocer `UsuarioDAOImpl` ni saltar capas hacia MySQL. Entonces, cuando el Service necesita guardar o buscar un usuario, ¿a qué le habla? Si no hay nada en el medio, volvemos al problema de la Práctica 6.

La respuesta: una interfaz

Le habla a `UsuarioDAO`: un contrato que dice *qué* se puede pedir (alta, búsqueda, etc.) sin decir *cómo* se hace por adentro. `UsuarioDAOImpl` es quien implementa ese contrato con Hibernate y MySQL. El Service depende del contrato; la implementación concreta queda abajo, reemplazable.

El nombre técnico de esa solución

Eso se llama **Principio de Inversión de Dependencias (DIP)**: los módulos de alto nivel (quien coordina: Resource y Service) no dependen de los de bajo nivel (`UsuarioDAOImpl`, Hibernate, MySQL). Ambos miran hacia una abstracción —en nuestro caso, la interfaz `UsuarioDAO`.

Antes (Práctica 6)

Alto nivel dependía directo del bajo nivel: `UsuarioResource` conocía `UsuarioDAOImpl`. Cambiar la implementación obligaba a tocar la puerta HTTP.

Ahora (Entrega 4)

`UsuarioService` depende de `UsuarioDAO`. `UsuarioDAOImpl` implementa la interfaz. Se puede cambiar cómo se persiste sin tocar Resource ni la regla de capas.

Conexión con la diapositiva 2

DIP no es un tema nuevo aparte. Es la herramienta concreta que permite cumplir la regla de “qué sí y qué no puede saber cada capa”: el Resource conoce al Service, el Service conoce la interfaz del DAO, y nadie de arriba importa `UsuarioDAOImpl`.

Inyección de dependencias: alguien te pasa lo que necesitas

en vez de crear la dependencia adentro

DI es la forma concreta de cumplir DIP: en lugar de que el Resource cree el DAO a mano, alguien desde afuera le entrega la dependencia ya lista.

Tres formas de inyectar

a) Por constructor

Las dependencias se pasan al crear el objeto. Es la más recomendada en general: deja claro qué necesita la clase y facilita los tests.

b) Por campo (field injection)

La dependencia se declara en un campo y el contenedor la completa. Es la forma más corta de escribir con CDI y Jersey.

c) Por setter

Se entrega la dependencia con un método de asignación. Sirve sobre todo cuando la dependencia es opcional.

Cómo Weld elige qué entregar

Con `@Inject`, Weld busca un bean compatible con el tipo pedido. Si pedís una clase concreta (por ejemplo el Service), entrega el bean de esa clase. Si pedís una interfaz (por ejemplo el DAO de usuario), busca cuál bean la implementa; si hay una sola, la entrega automáticamente. Eso conecta los dos lados de la idea DIP de la diapositiva anterior: DIP pone la interfaz en el medio; Weld es **quién conecta** el lado que pide el contrato con el lado que lo implementa, sin que el alto nivel conozca la clase concreta.

Por qué elegimos field injection en esta entrega: con CDI y Jersey es la forma más corta de escribirlo, y es lo que pedía el práctico. La inyección por constructor sería más testeable; field injection es lo que aplicamos acá.

Inversión de control: ya no manejamos el ciclo de vida

quién crea, inyecta y destruye

En la diapositiva anterior vimos cómo Weld encuentra el bean correcto cuando alguien pide un tipo (clase concreta o interfaz). Eso es solo una parte de lo que hace: además de elegir la pieza correcta, también decide **cuándo crearla** y **cuándo destruirla**, cosas que antes hacía el programador a mano.

Antes

El **programador** decidía cuándo y cómo se creaba cada objeto: en qué momento nacería el DAO, con qué clase concreta, y en qué línea del código.

Esa decisión se repetía en cada lugar donde hacía falta esa pieza (o se centralizaba a mano en un Factory, pero igual la inventaba el programador).

Ahora

Weld es quien toma esa misma decisión: cuándo crear el objeto, cuándo entregárselo a quien lo pidió, y cuándo destruirlo.

El programador solo declara qué necesita con `@Inject`, sin decidir el momento: no elige cuándo nace ni cuándo muere la pieza.

Por qué se llama “inversión”

El control que antes estaba del lado del programador ahora está del lado del framework. Con la diapositiva anterior (DI) vimos que Weld sabe **qué** bean darte (según el tipo pedido: clase o interfaz); acá se ve que Weld también decide **cuándo** dártela y cuándo destruirla. Son dos preguntas distintas —qué pieza y en qué momento— resueltas por la misma pieza.

Del “framework” al motor concreto

estándar · implementación · escaneo

La pregunta que queda de IoC

Dijimos que “el framework” decide crear, inyectar y destruir. Eso no es magia abstracta: hace falta un **estándar de reglas** (qué anotaciones existen y qué significan) y una **herramienta concreta** que las ejecute al arrancar y en cada petición.

CDI = el estándar

CDI es el contrato de Jakarta para inyección y ciclo de vida. Define anotaciones como `@Inject`, `@RequestScoped`, `@ApplicationScoped`, `@Produces` y `@Disposes`. Por sí solo no crea ni destruye nada: solo dice las reglas.

Weld = quien las cumple

Weld es la implementación concreta que usamos. Es quien de verdad lee esas anotaciones, crea los objetos, los inyecta y los destruye. Sin Weld, las anotaciones serían etiquetas sin efecto.

Cómo se entera Weld de qué clases administrar

Al arrancar, Weld recorre las clases del proyecto buscando cuáles tienen anotación de scope (`@RequestScoped` o `@ApplicationScoped`). A esas las registra como **bean**: objetos cuyo ciclo de vida gestiona él. Eso se habilita con `beans.xml` en modo `bean-discovery-mode="annotated"`: “escaneá solo lo anotado”. Sin ese archivo, Weld no arranca y `@Inject` no funciona.

Por qué el Resource también tiene que ser bean

Jersey atiende HTTP, pero no inventa las dependencias solo. Para poder inyectarle el Service al Resource, esa clase tiene que pasar por Weld. El puente entre Jersey y CDI es `jersey-cdi1x-servlet`. Si el Resource no fuera bean, Jersey lo crearía a mano y lo que debería inyectarse quedaría vacío.

Puente a la siguiente diapositiva: ya sabemos quién es el motor y cómo registra beans. Falta ver **cuánto dura** cada bean (por petición o por toda la app) y cómo una fábrica permanente puede fabricar algo distinto en cada pedido.

Cuánto dura cada bean (y la fábrica vs lo fabricado)

@RequestScoped · @ApplicationScoped · Producer

@RequestScoped — una instancia por petición

En un `POST /api/usuarios`, Weld crea para ese pedido un `UsuarioResource`, un `UsuarioService`, un `UsuarioDAOImpl` y el `EntityManager` de esa petición. En un pago, crea `PagoResource`, `PagoService` y los tres DAOs que coordina (`PagoDAO`, `InscripcionCongresoDAO`, `UsuarioDAO`), todos compartiendo el **mismo** `EntityManager` de esa petición puntual. Al terminar el pedido, **todas esas instancias se destruyen juntas**: no queda estado colgado para el próximo cliente.

@ApplicationScoped — una sola instancia para toda la app

Ejemplos: `EntityManagerProducer` y `DocumentStorageService`. Esos dos no necesitan renovarse en cada pedido porque **no guardan datos de una petición particular**: el `Producer` solo sabe cómo crear y cerrar `EntityManager`s; el servicio de documentos solo sabe guardar y leer PDFs en disco. Una instancia permanente alcanza para toda la vida de la aplicación.

¿No es contradictorio? Fábrica permanente, producto por petición

`EntityManagerProducer` es una clase única y permanente (`@ApplicationScoped`). Adentro tiene un método con `@Produces` que fabrica un `EntityManager` **distinto por cada petición**, y un método con `@Disposes` que lo destruye cuando esa petición termina. La fábrica es una sola; lo que fabrica es distinto cada vez. Por eso conviven `@ApplicationScoped` (la fábrica) y scope de request (el `EntityManager` producido) en el mismo mecanismo, sin contradicción.

Puente a la siguiente: ya se vio quién administra (Weld), qué reglas sigue (CDI) y cuánto dura cada bean (`@RequestScoped` / `@ApplicationScoped`). Falta ver cómo Weld decide **qué bean entregar** cuando se pide un tipo con `@Inject` (una clase concreta o una interfaz).

Qué implementación inyectar, y un EntityManager por petición

@Inject · @Produces · @Disposes

La pregunta que quedó abierta

Con `@Inject`, Weld busca un bean compatible con el tipo pedido: clase concreta (ej. `UsuarioService`) o interfaz (ej. DAO de usuario → `UsuarioDAOImpl` si es la única). El escaneo de scopes llena ese registro de beans.

¿Por qué un solo EntityManager por petición y no uno por DAO?

El problema — historia de un pago a medias

Imaginá que `PagoService` coordina tres DAOs y cada uno abre su propio EntityManager y su propia transacción. Primero el DAO de pago guarda el pago: todo bien, confirma y cierra. Después el DAO de inscripción guarda la inscripción: también bien, confirma y cierra. Al final el DAO de usuario falla (por ejemplo, un dato inválido). Los dos primeros **ya quedaron confirmados en MySQL** y no se pueden deshacer: hay un pago y una inscripción registrados, pero el resto de la operación no terminó. Quedó todo a medias. Lo que se perdió es la **atomicidad**: la garantía de que o se guarda **todo junto**, o **no se guarda nada**.

La solución: `EntityManagerProducer`

Con `@Produces` y scope de request, Weld crea **un** EntityManager al inicio de la petición y abre la transacción. Los tres DAOs del pago (`PagoDAO`, `InscripcionCongresoDAO`, `UsuarioDAO`) usan ese mismo EntityManager. Al terminar, `@Disposes` cierra el ciclo: si los tres funcionaron, hace **commit de los tres juntos**; si cualquiera falló, hace **rollback de los tres juntos** — y cierra el EntityManager. Así el pago no puede quedar a medias.

Puente: el Producer fabrica un EntityManager por petición, pero ¿quién deja lista la fábrica antes de que llegue el primer pedido? Eso no es un gancho de bean CDI: es el arranque de Tomcat. Siguiendo la diapositiva.

Fábrica una vez, EntityManager por petición

¿Por qué no @PostConstruct / @PreDestroy ?

Esos son ganchos de **CDI**, atados al ciclo de vida de un bean (@RequestScoped o @ApplicationScoped): corren cuando nace o muere ese bean. Acá hace falta algo que exista **antes** de que arranque cualquier bean y se cierre **después** de apagar todo el servidor. Tomcat ofrece su propio gancho: ServletContextListener , con contextInitialized al levantar y contextDestroyed al bajar. Eso es lo que usa JpaBootstrapListener , en vez del gancho de CDI.

EntityManagerFactory y EntityManager: suenan parecido, no son lo mismo

La **EntityManagerFactory** es pesada de armar: lee persistence.xml , prepara la conexión a MySQL, deja listo Hibernate. Se crea **una sola vez** para toda la vida del servidor: eso lo hace JpaBootstrapListener en contextInitialized , y la cierra en contextDestroyed .

El **EntityManager** es lo que esa fábrica **produce**: uno nuevo por cada petición HTTP. Eso lo hace EntityManagerProducer (@Produces / @Disposes), usando la fábrica que el listener ya dejó lista.

En una frase: JpaBootstrapListener arma la fábrica una sola vez; EntityManagerProducer usa esa fábrica para sacar un producto nuevo en cada petición (y destruirlo al terminar el pedido).

POST /api/usuarios : de la red al Service

Fases A, B y C

Fase A · Cliente y Jersey (igual que Práctica 6)

Alguien completa el formulario de alta de usuario y lo envía. Llega POST /api/usuarios con JSON. Tomcat lo deriva al ServletContainer de Jersey (web.xml , rutas /api/*). Jersey enruta con @ApplicationPath("/api") , @Path("/usuarios") y @POST . Jackson convierte el JSON en un objeto Usuario . Para llamar a alta(...) , Jersey necesita un bean CDI, no un new a mano.

Fase B · Weld crea e inyecta (nuevo en E4)

El servidor arma el equipo que va a atender este pedido puntual. Gracias a jersey-cdi1x-servlet (el puente Jersey–CDI que ya vimos), Weld (el motor CDI) entrega un UsuarioResource @RequestScoped (una instancia solo para esta petición). Ve @Inject UsuarioService y crea el Service; ve @Inject UsuarioDAO y crea UsuarioDAOImpl (clase concreta o interfaz: Weld elige el bean compatible, como en DI). El DAO pide un EntityManager : Weld llama al @Produces de EntityManagerProducer (la fábrica permanente que saca un EM por petición), abre la transacción, y ese mismo EntityManager lo comparten Resource, Service y DAO.

Fase C · Presentación delega en Aplicación

Antes de guardar, alguien tiene que revisar si el pedido tiene sentido. UsuarioResource.alta(...) no llama al DAO. Llama a usuarioService.alta(usuario) (el Resource solo conoce al Service de al lado, regla de capas). El Service valida: email no repetido, email en minúsculas, rol PARTICIPANTE por defecto. Si el email ya existe, lanza NegocioException .

Del DAO a MySQL: error 400 o éxito 201

Fases D y E

Fase D · Aplicación, infraestructura y MySQL

Ya se validó que el email es válido; ahora hay que guardarlo de verdad en la base. `UsuarioService` llama a `usuarioDAO.alta(usuario)`. En modo CDI, `AbstractJpaDAO` hace `persist` sobre el `EntityManager` inyectado (el mismo de esta petición; no abre otra transacción — atomicidad del `Producer`). Hibernate lee `@Entity` / `@Table` y arma el `INSERT`. MySQL asigna el `id`; el objeto en memoria ya lo tiene.

Fase E · Dos salidas posibles

Acá el sistema se pregunta: ¿el email que mandaron ya existía o no? Según la respuesta, el camino se bifurca.

Error de negocio

El email ya estaba: se rechaza el alta. `NegocioException` sube hasta Jersey. `NegocioExceptionMapper` (*ExceptionMapper: convierte la excepción en respuesta HTTP*) responde HTTP **400** con `{"error":"..."}`. `@Disposes` hace **rollback** y cierra el `EntityManager`: no queda fila a medias.

Éxito

El email era nuevo: se confirma el alta y se avisa al cliente. El Resource convierte el resultado a `UsuarioDTO` (la versión sin password que vimos en Arquitectura), responde HTTP **201** con header `Location`, Jackson arma el JSON, y `@Disposes` hace **commit** y cierra.

Detalle de coloquio

El Resource no arma a mano el JSON de error: lanza la excepción correcta y el `ExceptionMapper` hace el resto. Misma idea para 404 con `NotFoundExceptionMapper`.

Probar la API sin inventar otro camino

OpenAPI · misma petición real

Qué es

Swagger es la interfaz visual para probar los mismos endpoints que curl o Postman. **No ejecuta lógica de negocio propia:** dispara el flujo HTTP real (Tomcat, Jersey, Weld, Service, DAO, MySQL) y muestra el código de respuesta. La página está en `/swagger-ui/` (`swagger-ui/index.html`).

Qué hace cada anotación en la pantalla

@Tag

Agrupa endpoints bajo un nombre visible (Usuarios, Trabajos, Pagos...).

@Operation

Describe qué hace ese endpoint: es el título del formulario “Try it out”.

@ApiResponse

Documenta códigos HTTP posibles (201, 400, 404...) antes de probar.

Cómo se arma solo

`OpenApiResource` lee las anotaciones y genera `GET /api/openapi.json`. Swagger UI lee ese JSON y dibuja los formularios sola. Un endpoint nuevo bien anotado **aparece solo** en la pantalla.

Cómo comprobamos que persistió

En la misma página de Swagger: después del POST (“Try it out” → Execute), un **GET** del mismo recurso. Si volvió el JSON, persistió. Si fue 400, el GET no muestra esa fila. Sin phpMyAdmin.

Qué pedía la entrega (y cómo lo mostramos en Swagger)

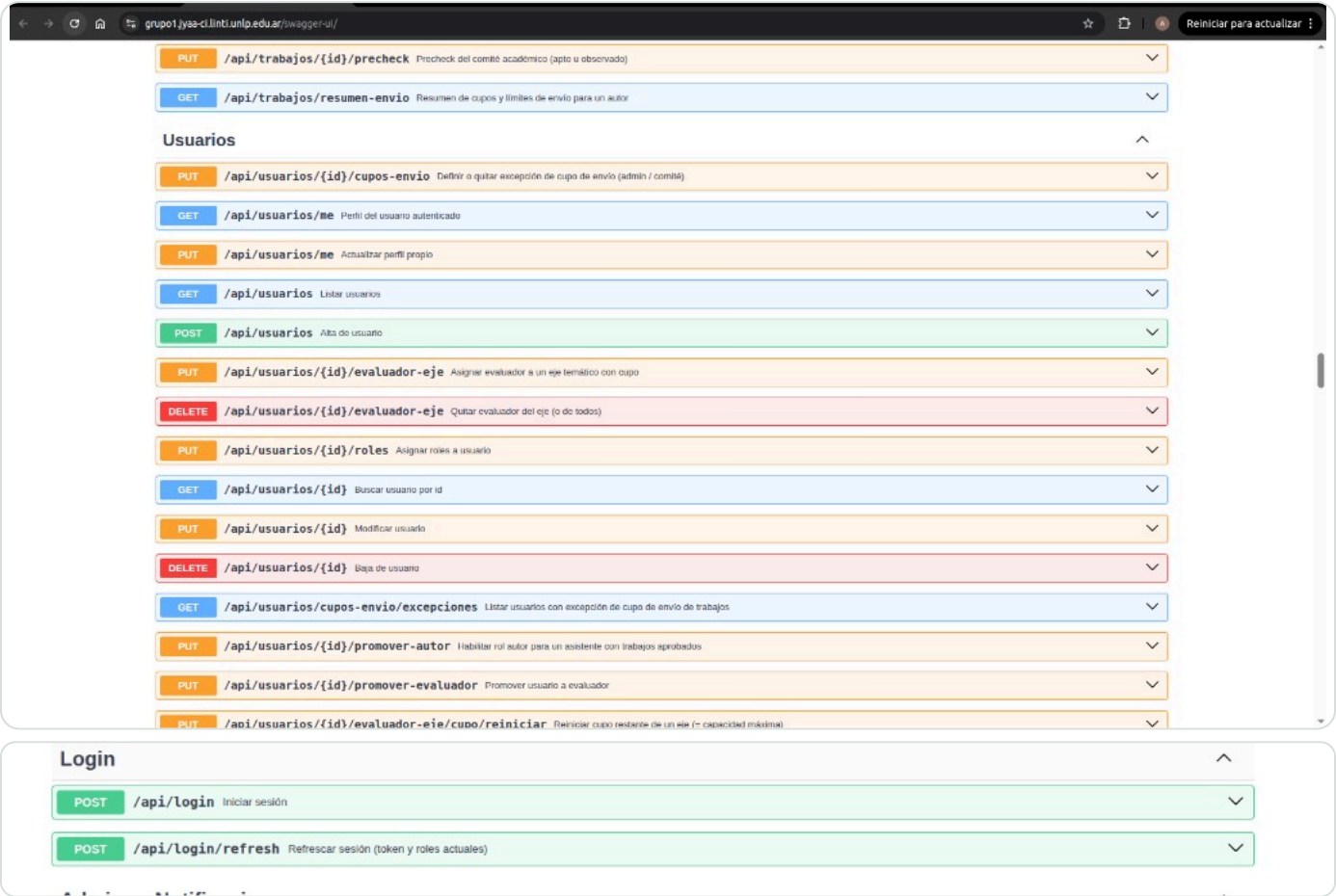
rama entrega-4

Lo más importante que pedía la Entrega 4

- API REST con Jersey: endpoints por rol (Admin, OC, Evaluador, Autor, Participante).
- Capas limpias con CDI: Resource → Service → DAO, sin `new` a mano.
- Persistencia real en MySQL del curso (los datos sobreviven al reinicio).
- Swagger / OpenAPI para documentar y probar la API en vivo.
- DTOs en la respuesta (sin password ni ciclos de Jackson).
- Archivos (PDF) asociados a trabajos/pagos: se guarda la URL, no el binario en la entidad.

Qué se ve en las capturas

La UI real en `/swagger-ui/` de nuestro servidor: el tag **Usuarios** con GET/POST/PUT/DELETE y sus `@Operation` (“Alta de usuario”, “Listar usuarios”...), y el tag **Login** con “Iniciar sesión” y “Refrescar sesión”. Sale solo de las anotaciones: nadie dibujó esa pantalla a mano.



Capturas reales de `grupo1.jyaa-ci.linti.unlp.edu.ar/swagger-ui/`: tags + operaciones documentadas; se prueba con Try it out y se verifica con GET.

Angular en el navegador, Java en el servidor

rama entrega-5

La Entrega 4 dejó una API REST funcionando. ¿Cómo hace una app que corre en el navegador del usuario para hablar con nuestro backend Java que vive en otro servidor?

Lo que ya teníamos (E4)

Ya teníamos el motor funcionando del lado del servidor: un programa Java corriendo en Tomcat que sabe recibir pedidos, buscar datos en la base y devolver respuestas — organizado en capas (Resources, Services, DAOs) y empaquetado en lo que se llama un WAR. Se podía probar ese motor con herramientas técnicas como curl o Swagger UI, pero todavía no había una pantalla real para que un usuario común lo use.

Lo que suma la E5

Una **SPA Angular**: una sola página que, sin recargar todo el sitio, muestra pantallas (login, admin, listados) y pide datos a nuestra API REST (las rutas que empiezan con `/api/...`). El prototipo maquettato de la E1 (React + Vite, con datos guardados en el navegador vía `localStorage`, sin backend real) servía para validar los flujos del congreso antes de tener persistencia. La E5 lo reemplaza por un cliente real en Angular que habla contra la API Java de la E4.

Hilo de esta entrega

Para que Angular y Java se entiendan hay que resolver un problema de seguridad del navegador (orígenes distintos), elegir cómo se conectan en desarrollo y en producción, y entender el camino de un clic hasta los datos en pantalla.

Por qué el navegador bloquea pedidos entre dominios

CORS · regla del navegador

CORS no es una regla de Angular ni de Java: es una regla de seguridad del **navegador**. Un **origen** es la combinación de protocolo, dominio y puerto.

Orígenes distintos

Origen A (frontend): `http://localhost:4200`

Origen B (API): `https://grupo1.jyaa-ci.linti.unlp.edu.ar`

No son el mismo origen. Si la página cargada en A quiere **leer** la respuesta de B, el servidor B tiene que decir explícitamente que A está autorizado (headers como `Access-Control-Allow-Origin`). Si no, el navegador bloquea aunque el servidor haya respondido bien.

Mismo origen

Un origen es protocolo + dominio + puerto — no importa la ruta que sigue después. La página en `https://grupo1.../` y la API en `https://grupo1.../api` comparten protocolo, dominio y puerto: son el mismo origen, aunque las rutas sean distintas. Ahí CORS no aplica: el navegador no ve un cruce de dominios. Eso es lo que buscamos en producción.

En una frase

Sin permiso explícito del servidor, el navegador no deja que el código de Angular use los datos que llegaron, aunque la respuesta haya sido correcta. El bloqueo aparece en la consola del navegador (DevTools) como “blocked by CORS policy”. Por eso la E5 tiene que decidir cómo conectar frontend y backend en cada entorno.

Desarrollo: dos formas de probar sin tocar producción

`npm start` vs `npm run start:cors`

Para programar día a día no queremos pelearnos con el problema de seguridad del navegador (CORS) que vimos en la diapositiva anterior. Por eso hay dos formas de levantar Angular en desarrollo: una lo evita, la otra lo enfrenta a propósito.

Forma 1 — con proxy (`npm start`)

Acá hay dos servidores: el de desarrollo de Angular (tu PC, `localhost:4200`) y el backend real (grupo1). El navegador solo le habla al primero — nunca se entera de que existe el segundo.

Angular pide los datos a una dirección simple, `/api/usuarios`, sin decir en qué servidor buscarla (`environment.ts`). El navegador la busca ahí mismo, en `localhost:4200`: nunca cruza a otro dominio, así que CORS no se activa.

El que sí cruza es el servidor de Angular: `proxy.conf.json` le dice “lo que llegue a `/api/...`, reenvíalo a grupo1 y traé la respuesta”. Ese salto lo hace por su cuenta, sin que el navegador participe.

Es la forma que usamos día a día: cómoda, pero no prueba que el backend sepa manejar un cruce de dominios real — acá nunca lo enfrenta.

Forma 2 — con CORS real (`npm run start:cors`)

Acá el servidor de desarrollo de Angular deja de ser intermediario: Angular pide los datos directo a la dirección completa de grupo1 (`environment.cors.ts`), sin pasar por ningún proxy.

Ahora es el navegador el que hace ese pedido, y ve dos direcciones distintas: `localhost:4200` (donde está la página) y grupo1 (donde está la API). Al detectar ese cruce, exige que el servidor confirme explícitamente que está permitido — eso es CORS real.

No es la forma cómoda para trabajar — la usamos a propósito para comprobar que el backend sabe responder ese permiso correctamente.

Ninguna de las dos es la que usa quien entra al congreso desde el entorno desplegado (grupo1) — esa es la Forma 3, que vemos más adelante. Antes, veamos cómo el backend resuelve ese permiso que la Forma 2 exige.

Cómo el backend resuelve el permiso CORS

preflight + los tres filtros Java (solo aplica a la Forma 2)

Cuando Angular usa la Forma 2 (CORS real) y hace un pedido “no simple” —como un POST con JSON, por ejemplo el login— el navegador no lo manda directo. Antes hace una pregunta de prueba: un `OPTIONS` que dice “¿me dejás que `localhost:4200` haga esto?”. Eso se llama **preflight**. Si la respuesta es sí, recién ahí manda el pedido real. Si falla, ni lo intenta.

Quién contesta esa pregunta y quién más interviene

Tres filtros Java, registrados junto al resto de la API en Jersey, sin tocar Resources ni Services:

- `CorsConfig` es la lista de invitados: guarda qué direcciones (`localhost:4200`, por ejemplo) tienen permiso.
- `CorsRequestFilter` es quien responde el preflight: si el método es `OPTIONS`, consulta la lista y contesta sí o no. Si dice que sí, recién ahí el navegador manda el POST real.
- `CorsResponseFilter` actúa después, cuando el Resource ya armó la respuesta real (el JSON del login, por ejemplo): le agrega el sello de autorización para que el navegador la deje leer.

Ejemplo completo con el login

El navegador pregunta con `OPTIONS`, `CorsRequestFilter` contesta que sí, el navegador manda el POST con el email y la contraseña, el Resource procesa el login, y `CorsResponseFilter` le agrega el sello final a la respuesta antes de que Angular la reciba.

Forma 3 — producción (el entorno desplegado del congreso)

`git push` · mismo origen · sin CORS

Después de ver cómo el backend resuelve el permiso CORS en desarrollo, toca ver el entorno donde ese problema directamente no aparece.

A diferencia de las dos formas anteriores, que solo sirven para desarrollar y probar en la máquina de quien programa, esta es la única que corre para quien usa el sistema del congreso en el entorno desplegado (grupo1): admin, organizador, autor, etc. abren esa URL pública, no `localhost`.

Cómo se despliega

Un `git push` a GitLab dispara el pipeline: Docker arma la imagen y Tomcat la despliega en grupo1. No se sube el WAR ya armado — se sube el código, y el WAR se arma en el servidor.

Cómo se arma el paquete final

El Dockerfile hace tres pasos: compila Angular (con la dirección de la API ya fija en `/api`), copia esos archivos dentro de la carpeta del backend (reemplazando su página de inicio), y Maven empaqueta todo junto en un solo WAR.

¿Hay CORS acá?

No. La página y la API viven en el mismo servidor (`grupo1.../` y `grupo1.../api`), así que el navegador nunca detecta un cruce de dominios.

Qué se buscaba con la integración

para qué servía esta entrega

1 · Cliente real contra la API real

Dejar de simular datos en el navegador. Cada pantalla relevante habla con endpoints de la Entrega 4 y respeta roles (admin, organizador, evaluador, autor, participante).

2 · Entender el cruce navegador / servidor

Que el grupo sepa por qué falla un pedido “que en Swagger anda” cuando Angular corre en otro origen, y cómo se resuelve (proxy, CORS o mismo WAR).

3 · Un camino a producción

Que frontend y backend puedan vivir juntos en el despliegue del curso: un push actualiza la app pública en grupo1, usable sin levantar npm en cada máquina.

En resumen: la E5 demuestra que la API de la E4 no es solo para Swagger: es consumible por una interfaz de congreso en el navegador, en desarrollo y en el servidor de la cátedra.

Del clic a la tabla: cómo Angular pide datos

HttpClient · Observable · servicios

Historia: listar usuarios (admin)

El administrador abre el listado de usuarios. La pantalla no habla con MySQL directamente: se lo pide a un servicio Angular (`UsuarioService`), que arma el pedido usando `HttpClient` (el cliente HTTP que Angular da automáticamente).

Un detalle importante: `listar()` no dispara el pedido al llamarlo. Devuelve un `Observable` — como una receta que todavía no se cocinó. Recién cuando la pantalla se suscribe, sale el GET real.

Camino completo

Todo arranca con el clic en la pantalla, que dispara `listar()`. `HttpClient` arma el GET (la dirección exacta depende de la forma: proxy, CORS o producción) y lo manda hacia Tomcat y Jersey. Ahí el pedido pasa por `UsuarioResource`, el Service y el DAO, hasta consultar MySQL. La respuesta vuelve como JSON, la suscripción la guarda, y recién ahí Angular repinta la tabla.

Conceptos de Angular detrás de esta historia

Para que este camino funcione entran en juego varias piezas de Angular: el componente que muestra la pantalla, la inyección de dependencias (Angular le entrega el servicio ya armado, sin que el componente tenga que crearlo), el router y los guards (que veremos más adelante: deciden si el admin puede llegar a esta pantalla), y `HttpClient` con `Observables/RxJS` para el pedido en sí.

Qué no cambia entre las tres formas

El recorrido interno (servicio, `Observable`, suscripción, tabla) es el mismo. Solo cambia cómo llega el HTTP a la API.

Qué conceptos de Angular se ven en esta entrega

del lado del navegador, hablando con Java

Inyección de dependencias, versión Angular

En Java usamos CDI: le pedís a la clase que necesitás y el framework te la entrega ya armada, sin que vos la construyas a mano (`@ApplicationScoped` , por ejemplo). Angular hace exactamente lo mismo con sus propias palabras: un servicio marcado `@Injectable({ providedIn: 'root' })` se convierte en un único objeto compartido en toda la app, y cualquier componente que lo necesite lo recibe armado con solo pedirlo en su constructor. Es la misma idea de fondo (no construir vos mismo tus dependencias) aplicada del otro lado del proyecto.

Router

Angular es una SPA: una sola página que nunca se recarga. El router es quien decide qué pantalla mostrar según la URL (login, selección de rol, panel de admin, listado de usuarios), sin pedirle un nuevo HTML al servidor cada vez — solo intercambia componentes en el lugar de la pantalla que cambia.

Guards

Antes de dejar entrar a una ruta, el router primero le pregunta a un guard: “¿este usuario puede estar acá?”. En este proyecto se usan para dos cosas: confirmar que hay una sesión iniciada (`authGuard`), y confirmar que el rol del usuario coincide con el que exige esa pantalla (`roleGuard` , por ejemplo para que solo un admin entre a `/admin`).

HttpClient y Observables (RxJS)

Es la herramienta que arma y envía los pedidos HTTP hacia la API Java. No trae el dato al instante: devuelve un Observable, que recién se activa cuando la pantalla se suscribe — el mismo concepto que vimos en la diapositiva de Consumo.

Ninguno de estos conceptos vive aislado: la inyección de dependencias conecta directamente con CDI del lado Java, y el router junto con los guards son quienes deciden si un pedido llega a dispararse.

Qué conceptos de Java web se ven en esta entrega

del lado servidor, al servicio de Angular

JAX-RS y Jersey

JAX-RS es el estándar para APIs REST en Java; Jersey es la implementación que usamos. Define cómo una URL como `/api/usuarios` llega a un método Java. Angular no “entra” a Tomcat ni ejecuta código Java: se comunica con Jersey de la única forma que puede un programa externo — mandando pedidos HTTP normales (GET, POST, etc.) a esas URLs, igual que lo haría un navegador cualquiera. Jersey recibe ese pedido, lo enruta al método Java correcto, y devuelve la respuesta en JSON. Así es como Angular consume la API REST del backend, sin necesitar saber cómo está escrito el Java del otro lado.

Filtros (`@Provider`)

Piezas de código Java que envuelven cada pedido o cada respuesta, sin mezclarse con la lógica del Resource. En este proyecto hay tres, todos relacionados con CORS: `CorsConfig` (guarda qué orígenes tienen permiso para pedir datos), `CorsRequestFilter` (contesta la pregunta previa del navegador, el preflight) y `CorsResponseFilter` (agrega el sello de autorización a la respuesta real antes de que Angular la reciba). Son el ejemplo concreto de esta entrega de cómo resolver un problema transversal sin tocar el código de negocio.

Manejo de peticiones HTTP

Cada acción del usuario se traduce en un tipo de pedido HTTP distinto: GET para pedir datos (como listar usuarios), POST para crear algo nuevo o iniciar sesión, PUT para modificar, y DELETE para borrar. Angular elige el método según la acción, Jersey lo recibe y lo redirige al método Java correcto, y la respuesta siempre vuelve en JSON, un formato de texto ordenado que Angular puede leer fácilmente.

En producción hay un detalle extra: el archivo `web.xml` le dice a Tomcat que todo lo que empiece con `/api/` es para Jersey, y todo lo demás son pantallas de Angular. Como Angular maneja rutas internas (como `/admin/usuarios`) que no existen como archivos reales en el servidor, si alguien entra directo a esa URL Tomcat no la encuentra — pero en vez de mostrar un error, la redirige a la página principal de Angular, que ahí sí sabe armar esa pantalla. Ese es el “truco del 404”.

Estos tres conceptos no aparecieron de golpe en la E5: son la continuación de un mismo camino en Java web. Los Servlets simples de la E3 dieron paso a JAX-RS y Jersey como forma más ordenada de mapear URLs a métodos Java. La E4 sumó CDI para que esas clases no armaran sus propias dependencias a mano — la misma idea que Angular aplica con `@Injectable`, como vimos en la diapositiva anterior. Y ahora, en la E5, ese mismo mecanismo de filtros que ya usaba Jersey se aprovecha para resolver un problema nuevo: dejar pasar (o no) pedidos que vienen de otro origen, sin tocar el código de los Resources.

Qué hace cada pieza en el flujo

no solo nombres · el rol concreto

Desde el usuario: un pedido real (ej. registrarse)

Alguien completa el formulario de registro en Angular y aprieta enviar. Angular arma ese pedido (un POST) y HttpClient es quien lo manda por HTTP hacia el servidor — el intermediario entre la pantalla y la red. Ese pedido llega a Jersey, que lo enruta al método Java correcto (y de paso, si hace falta, pasa antes por los filtros CORS). El backend procesa el registro y la respuesta vuelve como JSON, que Angular usa para actualizar la pantalla.

Desde el desarrollo: cómo se llega a que eso funcione

Para que ese pedido tenga a dónde llegar, primero hay que empaquetar todo. Maven es quien compila el código Java del backend. Docker se encarga de todo el proceso de build: compila Angular por su lado, fusiona esos archivos con el backend, y ahí es cuando Maven arma el WAR final con todo junto, empaquetado dentro de una imagen lista para desplegar. Tomcat 10 es quien ejecuta ese WAR: sirve al mismo tiempo la SPA (las pantallas) y la API (los datos), en un solo proceso.

Ese paquete llega al servidor mediante GitLab CI: cada push dispara la construcción de la imagen y redespiega el contenedor de grupo1 con el código nuevo. Y para que alguien pueda entrar desde afuera, Traefik es quien publica la URL del grupo y enruta el tráfico HTTPS hacia ese contenedor.

Para programar sin tocar ese entorno

`npm` en desarrollo (`npm start` o `npm run start:cors`) levanta Angular en la PC, contra el backend real de grupo1, sin pasar por todo este pipeline.

Qué nos frenó y cómo lo resolvimos

CORS · deploy · WAR único

① CORS bloqueaba pedidos que “andaban” en Swagger

En Swagger todo parecía funcionar: la página de prueba y la API viven en el mismo servidor. Cuando Angular corría en la PC (`localhost`), el navegador veía dos direcciones distintas y bloqueaba la respuesta, aunque el backend hubiera contestado bien. **Cómo lo resolvimos:** tres caminos según el entorno — proxy en desarrollo (`npm start`), filtros CORS + `npm run start:cors` para probar el cruce real, o mismo origen en producción (página y API juntas en grupo1).

② Tener el código CORS en la PC no alcanza

grupo1 no lee la carpeta de la máquina de quien programa: corre lo último que desplegó el pipeline. Si los filtros CORS estaban solo en el repo local y no se había hecho push, `start:cors` fallaba aunque los archivos `.java` estuvieran bien escritos. **Cómo lo resolvimos:** subir el backend a GitLab y esperar el pipeline en verde antes de probar el cruce real.

③ Fusionar Angular y backend en un solo WAR

En el repo local, la carpeta del backend no muestra los archivos de Angular listos para servir: la unión no se hace a mano en la PC. Ocurre en el build de Docker: se compila Angular, se copia dentro del backend y Maven arma un solo paquete (WAR) con pantallas y API juntas. **Cómo lo resolvimos:** un Dockerfile en varias etapas — build de Angular, copia al `webapp` , empaquetado con Maven y despliegue en Tomcat como raíz del sitio.

Cierre E5: el navegador dejó de ser un obstáculo y pasó a ser el cliente oficial de la API. Con eso el sistema ya se puede usar como interfaz del congreso; la Entrega 6 cierra el producto completo en el entorno desplegado.

Sistema completo: del alta al certificado, en producción

rama de integración · producto usable en grupo1

La Entrega 5 dejó Angular hablando con la API Java. La Entrega 6 cierra el **producto del congreso**: flujos de punta a punta, seguridad de acceso, configuración por roles y el despliegue estable en el servidor de la cátedra.

Qué ya teníamos (E5)

SPA Angular contra la API, CORS/proxy/producción, y el camino de un clic hasta MySQL. Todavía faltaba el ciclo de negocio completo y que solo quien corresponde pueda operar cada pantalla.

Qué cierra la E6

Inscripción, pago, envío y evaluación de trabajos, programa, circulares, notificaciones, cierre y certificados — con JWT, más endpoints y pantallas configurables para administrador y comité académico.

En una frase

Pasar de “la API se puede consumir desde el navegador” a “el congreso se puede gestionar de punta a punta en grupo1, por cada rol, sin redeployar por cada edición”.

Qué se puede hacer en el sistema (por rol)

ciclo de vida del congreso

Administrador / organización

Desde la interfaz configura el congreso sin tocar código: **datos** (nombre, edición, sede y ubicación en el mapa) · **jornada** (días y horarios del evento) · **ventanas de tiempo** (cuándo se puede inscribir alguien) · **cupos** de inscripción (cuántos autores y asistentes acepta la edición) · define **aranceles por categoría** (con QR de pago) · valida pagos, emite **recibo correlativo**, hace **arqueo diario** y factura al participante · carga aulas con su ubicación en un **mapa interactivo** (librería Leaflet, estilo Google Maps: pinchos para ver a dónde ir), franjas y el programa · publica circulares y avisos · ve estadísticas, checklist pre-congreso y limpieza de archivos · **cierra el congreso** y habilita certificados.

Comité académico / organizador científico

Edita **catálogos** (ejes, modalidades, tipos de envío) · define plazos de envío y evaluación · gestiona **cupos de evaluador por eje** (se consumen y liberan al asignar, cancelar o dictaminar) · excepciones de cupo de envío · panel de comité: precheck, asignación de evaluadores, solicitudes, estadísticas · si hay empate 1–1, el sistema dispara un **tercer evaluador**.

Autores, evaluadores y participantes

Registro e inscripción · pago (transferencia o efectivo) con comprobante · envío de trabajos (artículo/relato), coautores, PDF · evaluación y devoluciones · agenda personal sin chocar horarios · propuestas de taller · descarga de certificado al cierre. Notificaciones **in-app y por email** según plantillas.

Público (sin sesión)

Home, historia del congreso, programa con mapa de aulas en Leaflet (ubicación en el mapa para saber a dónde ir), circulares públicas y configuración visible del evento — sin token JWT.

JWT, roles y los endpoints que faltaban

respecto de la E5

JWT — quién puede entrar a cada cosa

En la E5 ya había login en Angular: al iniciar sesión se guardaba el usuario de forma temporal en el navegador (`sessionStorage`), para no perder la pantalla al refrescar. Eso alcanzaba para “acordarse” de quién eras en el cliente, pero la API todavía se podía probar abierta (Swagger, curl) y el servidor no exigía un pase firme en cada pedido. En la E6 cerramos ese hueco: hay que **demostrar quién sos** también del lado Java.

Al iniciar sesión, el servidor no arma una sesión clásica en memoria de Tomcat: genera un **pase digital firmado** — un JWT — con el email y los roles (admin, autor, evaluador, etc.). Angular lo guarda en `sessionStorage` (junto con los datos del usuario) y lo manda en cada pedido a la API, en el encabezado `Authorization: Bearer ...`. Eso lo hace un **interceptor**: un intermediario que Angular aplica solo, sin que cada pantalla copie el token a mano.

Del lado Java, un **filtro** (`JwtAuthFilter`) revisa ese pase en casi todos los `/api/...`. Si falta o es inválido, el pedido no llega al Resource. Quedan libres solo cosas públicas: login, registro, health, OpenAPI/Swagger y algunas lecturas (programa, circulares, etc.). Además, en cada pedido el filtro **vuelve a mirar roles y si la cuenta sigue activa en MySQL**: así, si a alguien le cambian el rol o lo deshabilitan, no alcanza con tener un token viejo “congelado” en el navegador.

En Angular, antes de abrir una pantalla, los **guards** preguntan: ¿hay sesión (`authGuard`)? ¿el rol alcanza para esta ruta (`roleGuard` , por ejemplo solo admin en `/admin`)? Si no, redirigen al login o a otra pantalla. También hay un control de **inactividad**: si pasan 30 minutos sin usar la app, se cierra la sesión por seguridad.

API ampliada (respecto de la E4/E5)

Lo que vimos en la diapositiva anterior (admin, comité, autores, público) no son solo pantallas: cada flujo tiene sus endpoints REST en Jersey. Respecto de la E5 —centrada en conectar Angular con la API — acá el mostrador creció para cubrir todo el ciclo del congreso, no solo el CRUD de usuarios.

Configurable sin redeploy

Etapas, catálogos, aranceles, cupos y plazos los define el organizador o el comité desde la interfaz. Si estuvieran fijos en el código, cada edición del congreso pediría un despliegue nuevo.

Un solo WAR en grupo1, verificado en el pipeline

`git push` · mismo origen · sin CORS

Integración final

Con un `git push`, el pipeline arma todo solo. Un **Dockerfile** en varias etapas hace el trabajo: primero compila Angular (las pantallas), después las mete adentro del backend, Maven empaqueta pantallas y API en un solo archivo (el WAR) y Tomcat 10 lo ejecuta en grupo1. Como la página y la API viven en la misma dirección, en producción **ya no hace falta CORS**: para el navegador es un solo origen.

Dificultad: a veces salía un Angular viejo

Docker a veces reutilizaba capas guardadas de un build anterior y el contenedor de grupo1 quedaba con pantallas desactualizadas, aunque el código nuevo ya estuviera en GitLab. **Cómo la resolvimos**: forzar un build limpio (`--no-cache --pull`) y, sobre todo, una **verificación automática** dentro del Dockerfile: si el WAR no trae los archivos de Angular esperados (el `main-*.js` y los estilos), la imagen ni se publica. El error aparece en el pipeline, no delante del tribunal en la demo.

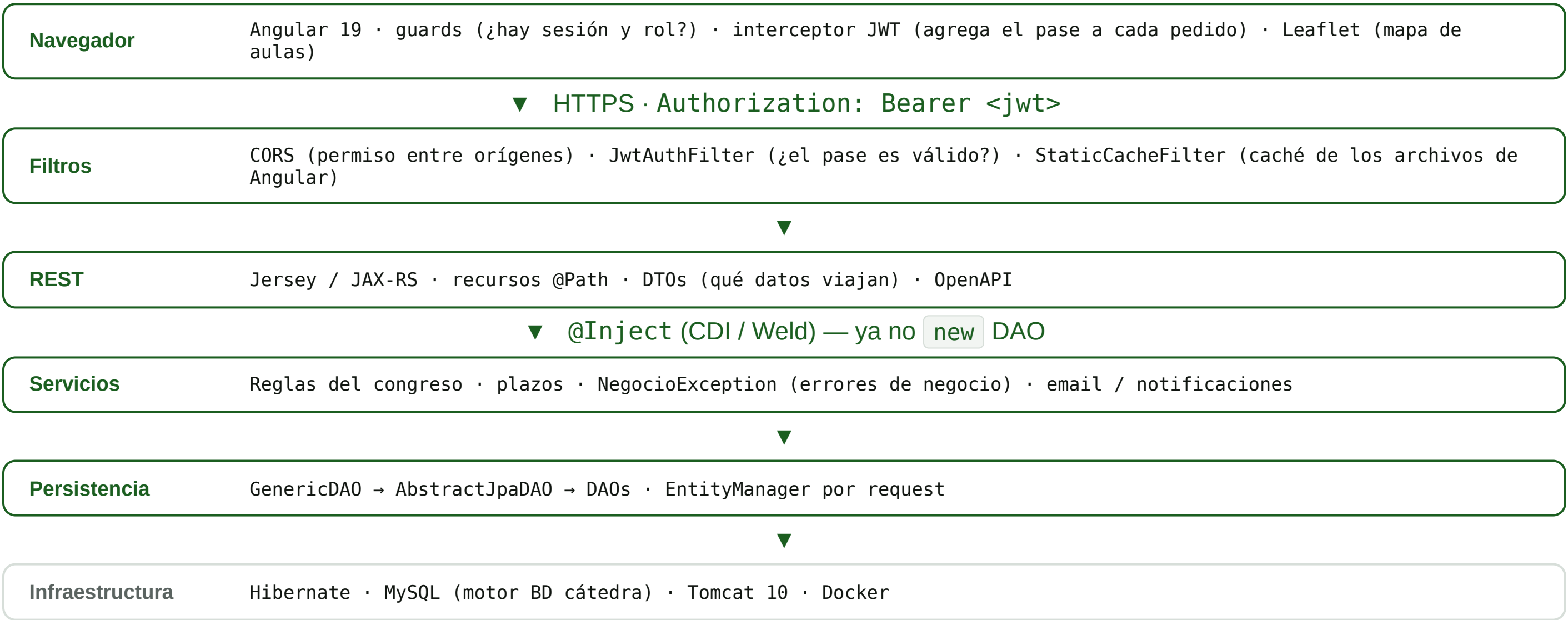
Cierre E6: el congreso ya se gestiona de punta a punta en el entorno desplegado. La app en vivo se muestra en el Bloque 4 (grupol.jyaa-ci.linti.unlp.edu.ar). Lo que sigue en el coloquio es la arquitectura resultante y cómo se organizó el trabajo del grupo.

La arquitectura que resultó de las seis entregas

Un solo WAR

Un pedido (por ejemplo listar usuarios o iniciar sesión) no “salta” al MySQL de una: baja por capas. Cada capa hace una sola cosa. En la E3 todavía estaban mezcladas (el Servlet hablaba directo con el DAO). En la E4 las separamos con CDI. En la E5 y la E6 sumamos Angular, filtros CORS/JWT y el WAR único en grupo1. Hoy el camino queda así:

Ningún tema de la cursada quedó sin usar: todos terminaron integrados en este mismo WAR.



03

Cómo trabajamos

La organización interna del grupo y las herramientas que la sostuvieron.

Nos repartimos por flujo del congreso, no por capa

3 integrantes

Al principio nos dividimos por capas —uno el backend, otro el front— y chocábamos todo el tiempo en los mismos archivos. A partir de la Entrega 3 cambiamos: **cada uno se hizo dueño de un flujo completo del congreso**, desde la entidad en Java hasta la pantalla en Angular.

Benjamín

El camino del participante que se inscribe y paga: inscripción, aranceles, validación de pagos, recibos, arqueo de caja y facturas. También la gestión de usuarios y gran parte del modelo de datos y de la API REST.

Alcides

El camino académico: envío de trabajos, comité, asignación de evaluadores, dictámenes y desempates. Además el programa del congreso: aulas, horarios, cronogramas y el mapa para ubicar las salas.

Lucas

Quién puede entrar y qué puede hacer (login, JWT, roles), certificados al cierre, circulares y notificaciones, y que el sistema quede desplegado: pipeline, Docker y pruebas de integración.

Lo que funcionó

Ser dueño de un flujo de punta a punta bajó casi a cero los choques al unir el código, y obligó a cada uno a tocar todas las capas (base, API y pantallas).

Lo que nos costó

Algunas piezas las usan los tres (por ejemplo el usuario y la configuración del congreso). Esas las hablamos antes de codear y las integramos primero, para no trabarnos entre nosotros.

Git, GitLab, Docker y despliegue continuo

un push actualiza la app en grupo1

- **Dos lugares para el código:** GitHub para trabajar día a día entre nosotros, y el **GitLab de la cátedra** para entregar y desplegar en el servidor del curso.
- **Una rama por entrega** (entrega-3 ... entrega-5) y ramas chicas por persona. La rama `main` siempre tiene que poder desplegarse: si el pipeline no está en verde, no se publica.
- **Docker arma el paquete en etapas:** primero compila Angular, después Maven mete pantallas y API en un solo WAR, y al final queda una imagen con Tomcat 10 lista para correr en grupo1.
- **GitLab CI** (con el runner `jyaa`) hace el trabajo solo al hacer push: construye la imagen, baja el contenedor viejo y levanta el nuevo. Traefik publica la URL HTTPS del grupo sin configurar certificados a mano.
- **Antes de publicar, se verifica el WAR:** el Dockerfile comprueba que adentro estén los archivos de Angular esperados. Si faltan, la imagen no sale.

Git

GitLab CI

Docker

Traefik + HTTPS

Maven

Eclipse / VS Code

phpMyAdmin

Swagger UI

Postman / DevTools

El pipeline, en cuatro pasos

- **1 · Armar** — se construye la imagen de cero (sin reusar caché vieja): Angular fresco + WAR verificado.
- **2 · Frenar** — se detiene el contenedor `grupo1` que estaba corriendo.
- **3 · Limpiar** — se elimina ese contenedor viejo.
- **4 · Publicar** — se levanta el contenedor nuevo y Traefik lo deja accesible en `grupo1.jyaa-ci.linti.unlp.edu.ar`.

Status	Pipeline	Created by	Stages	Actions
Passed	#14787 Hub Mis certificados: auto al cerrar cong... JF main => 315486796 branch		✓ ✓ ✓	↓
Passed	#14791 Mejorar altas de actividades: menos sobr... JF main => 315486796 branch		✓ ✓ ✓	↓
Passed	#14788 Dictamen del evaluador: tipo y modalida... JF main => 31348722 branch		✓ ✓ ✓	↓
Passed	#14787 Fix comite: NG0203 al cargar ejes y lista... JF main => 31498544 branch		✓ ✓ ✓	↓
Passed	#14786 Catálogos configurables de ejes, modali... JF main => 315486796 branch		✓ ✓ ✓	↓
Passed	#14782 Mostrar ocupación X/capacidad del aula ... JF main => 313486348 branch		✓ ✓ ✓	↓
Passed	#14769 Empale 1/1: el trabajo sigue en evaluaci... JF main => 82786496 branch		✓ ✓ ✓	↓

Pipelines de `main` en verde: cada push reconstruye y publica grupo1.

04

Demo y reflexión

El sistema funcionando de punta a punta, y qué nos llevamos del proceso.

Un trabajo, de principio a fin

6 min · grupo1.jyaa-ci.linti.unlp.edu.ar

Recorrido en <https://grupo1.jyaa-ci.linti.unlp.edu.ar/> — no pantallas sueltas: una acción de un rol cambia lo que ve otro, sobre la misma base.

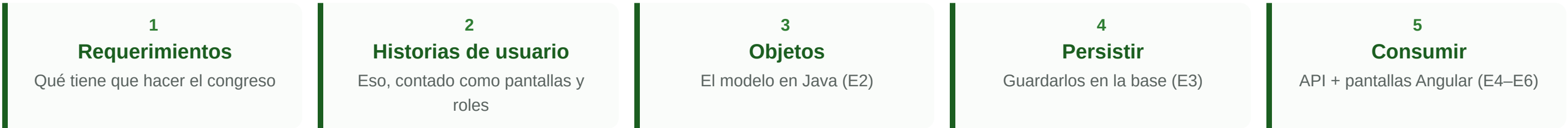
El flujo (en orden)

1. **Público** — home, programa con mapa de aulas y circulares (sin login).
2. **Asistente** — se registra, se inscribe y paga (transferencia + comprobante).
3. **Admin** — aprueba el pago; sale el recibo y llega la notificación.
4. **Autor** — carga el trabajo (coautores + PDF) y lo envía.
5. **Comité** — precheck y asigna dos evaluadores (respetando el cupo del eje).
6. **Evaluador** — acepta y dictamina.
7. **Autor** — ve la devolución en su panel.
8. **Admin** — cierra el congreso; se habilitan los certificados.

Qué nos llevamos

2 min

No fue “aprender una herramienta suelta”: fue **armar una aplicación de punta a punta**, en el mismo orden en que se construye un sistema real.



Y en paralelo

Desde temprano usamos **GitLab, Docker y el pipeline** para publicar en grupo1. Así cada avance no quedó en la notebook de alguien: quedó en el servidor, visible para todos.

Si hubiera una entrega más: contraseñas hasheadas, permisos por rol en el servidor, validar datos en la API y tests automáticos en el pipeline.

Gracias. ¿Preguntas?

Sistema de gestión del V Congreso Argentino de Agroecología

Aplicación: grupo1.jyaa-ci.linti.unlp.edu.ar **API:** </api/health> · **OpenAPI:** </api/openapi.json>

Repositorio: [GitLab de la cátedra — grupo 1](#)